DBMS PROJECT REPORT

Chandigarh University Academic Opportunity Tracker

Opportunity Compass

Database design and secure application implementation

November 2026

Abstract

The Academic Opportunity Tracker is designed to help Chandigarh University students organize academic opportunities and prepare for applications. Its relational model connects student profiles, departments, skills, organizations, opportunities, eligibility rules, applications, documents and preparation tasks. The current Documents application implements the foundational account and academic profile workflows, supported by a MySQL database and a Flask backend.

This report explains the problem, requirements, architecture, entity relationships, normalization choices and implemented routes. It also records the defensive security improvements: CU identity validation, password hashing, CSRF protection, current account and role checks, revocable sessions, rate limiting, narrow database permissions and safe error handling.

Verification completed on 3 November 2026 recorded 16 passing automated tests and eight successful live HTTP checks. Opportunity discovery, eligibility scoring, application processing and administrative management remain future modules in this application. The database supports their later implementation, but their presence in the schema does not mean the workflows are operational.

Project type Web based database management system
Platform Python Flask and MySQL 8
Current application http://127.0.0.1:5000

Contents and project overview

Section	Page
Problem statement and requirements	3
System architecture and data flow	4
Relational design and normalization	5
Student and identity data dictionary	6
Opportunity and application data dictionary	7
Documents operations and security tables	8
Implemented workflows and validation	9
Security design and audit improvements	10
Testing and verification results	11
Deployment limitations and future work	12

Project overview

Opportunity Compass provides the account and profile foundation for a campus opportunity planner. A student creates a university identity, signs in using a UID or email, and supplies academic information. These records can later support eligibility evaluation and application preparation. Administrators currently have an authenticated database availability check and a safe local account creation method.

The implementation described here is CampusOpportunityPlanner under the Documents directory. It is the application selected for review and served on port 5000. The separate Desktop DBMS application is not part of this report.

Current implementation boundary

The delivered application has six endpoint paths: home, registration, login, academic profile setup, logout and database diagnostic. It contains 19 original domain tables and two security tables. The landing page uses a static readiness illustration; its percentage is not calculated from a student record. Existing navigation placeholders do not provide the planned portal functions.

Problem statement and requirements

Problem statement

Students may encounter internships, scholarships, research projects and competitions through different channels. Comparing requirements and preparing documents can become difficult when identity, academic records and application information are disconnected. The project proposes a common relational foundation so each opportunity and application can be linked to the correct student and its associated requirements.

Objectives

The immediate objective is to maintain reliable student identity and academic profile data through a usable web interface. The broader objective is to support opportunity discovery, explainable eligibility, readiness preparation and application tracking through normalized relationships. Security controls must protect these records and prevent the browser from choosing an account role or record owner.

Requirement	Current status
Student registration and UID or email login	Implemented
Academic profile setup with validation	Implemented
Session logout and current role enforcement	Implemented
Administrator database availability check	Implemented
Opportunity discovery and eligibility evaluation	Planned
Applications documents tasks and notifications	Schema prepared only
Admin management screens and analytics	Planned

Nonfunctional requirements

The application uses parameterized SQL, transactions and database constraints to protect integrity. Errors must not reveal SQL, credentials or debugger output. Forms must use server validation and CSRF tokens. Existing data must survive additive migrations. The interface should remain responsive, with clear labels, consistent styling and readable feedback.

Users and operating environment

Students use the account and profile forms. An administrator can access the restricted diagnostic; broader administrative workflows are future work. The reviewed environment uses Flask 3.1.3, MySQL Server 8.0.46, MySQL Connector Python 26.10.0, Werkzeug 3.1.9, python dotenv 1.2.4 and Waitress 3.0.2.

System architecture and data flow

The system follows a server rendered web architecture. HTML templates define pages, CSS provides the existing navy and teal visual theme, and Flask handles requests. MySQL stores persistent relational records. The browser submits input but does not decide authentication, role, ownership or academic validity.

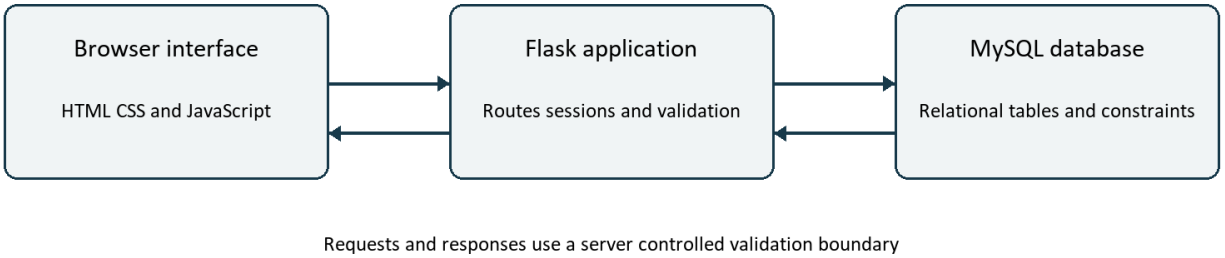


Figure 1 Application architecture

Request processing

A request first passes trusted host and environment checks. Unsafe methods must supply a matching CSRF token. For an authenticated request, Flask verifies the signed cookie and looks up its token digest in auth_sessions, then rechecks the current user and active status. Role guards protect the student profile and administrator diagnostic. Login and registration requests also pass through shared rate limits.

A route validates its fields and executes parameterized queries. Multi step changes run in a transaction: success commits, while an error rolls back. Cursors close after queries, and request teardown rolls back any unfinished work and closes the connection. READ COMMITTED isolation ensures statements see newly committed data rather than an old transaction snapshot.

Code responsibilities

Component	Responsibility
app.py	Existing route handlers and account or profile workflows
security.py	CSRF roles session lifecycle limits headers and errors
database.py	Connection query execution transaction and cleanup helpers
templates and static	Server rendered interface and stylesheet
scripts and tests	Migration administration verification and safe fixtures

Relational design and normalization

The schema separates identity, academics, reference data, opportunity requirements and application activity. A primary key identifies each entity. Foreign keys enforce valid references, while junction tables represent many to many relationships without storing comma separated lists.

Parent	Relationship	Child
users	One to zero or one	student_profiles
departments	One to many	student_profiles
student_profiles	Many to many through student_skills	skills
organizations and categories	Each one to many	opportunities
opportunities	One to zero or one	eligibility_requirements
students and opportunities	Many to many through applications	applications
applications	One to many	application_status_history
opportunities and documents	Many to many through opportunity_documents	document_types
applications	One to many	preparation_tasks

Normalization

First normal form is supported by atomic columns and separate rows for repeated facts. Student skills, interests, allowed departments and document requirements are represented as relationships. In the junction tables, the combined key identifies one relationship; attributes such as proficiency or requirement type describe that complete relationship, supporting second normal form.

Reference facts are separated from operational records. Department names belong to departments, organization details to organizations, and category descriptions to opportunity_categories. Student profiles and applications reference their keys instead of repeating these descriptions. This limits update anomalies and follows the intent of third normal form. A formal dependency proof would require documenting all business dependencies as the product grows.

Integrity and transactions

UNIQUE constraints protect email, UID, one profile per user and one application per student and opportunity. CHECK constraints bound academic values and selected dates. Foreign keys reject orphan references. Concurrent registration or profile requests rely on database uniqueness plus safe conflict handling. Existing cascade rules remain and need a retention review before deletion workflows are introduced.

Student and identity data dictionary

The following tables define student identity and academic classification. PK denotes primary key and FK denotes foreign key. Nullable academic fields allow the current setup form to omit CGPA or graduation year.

Table	Important fields	Purpose and constraints
users	user_id PK; full_name; email; password_hash; role; is_active; created_at	Central account. Email is unique. Role is student or admin. Passwords are hashes.
student_profiles	student_id PK; user_id FK; university_uid; department_id FK; current_year; cgpa; graduation_year	One profile per user and unique UID. Year 1 to 4 and CGPA 0 to 10 when present.
departments	department_id PK; department_code; department_name	Department code and name are separately unique.
skills	skill_id PK; skill_name; skill_category	Unique skill name and optional classification.
student_skills	student_id FK; skill_id FK; proficiency_level; is_verified	Composite PK on student and skill. Proficiency is beginner intermediate or advanced.
student_interests	student_id FK; category_id FK; interest_level	Composite PK links students with categories and an interest level.

Identity rules

A new student UID contains two intake digits, two to five program letters and four to six numeric digits. For example, 25bit10108 corresponds to 25bit10108@cuchd.in. Values are converted to lowercase before insertion. Registration accepts only the matching CU email and creates a student role regardless of extra submitted fields.

Registration inserts the user account first. The academic setup route later creates the profile, retaining the original two stage workflow. Before that profile exists, UID login can resolve the equivalent CU email. After setup, the profile UID supports lookup. A pre-existing non CU account is not silently renamed; legacy identity correction requires a trusted administrative process.

Opportunity and application data dictionary

These domain tables are present in the database design. Their management and application workflows are not yet implemented in the Documents application.

Table	Keys and selected fields	Design role
organizations	organization_id PK; organization_name; organization_type; website_url; contact_email	Maintains the organization offering an opportunity.
opportunity_categories	category_id PK; category_name; description	Classifies internship scholarship research and other listings.
opportunities	opportunity_id PK; organization_id FK; category_id FK; created_by FK; title; mode; application_deadline; listing_status	Stores listing details. Status is draft published closed or cancelled.
eligibility_requirements	opportunity_id PK and FK; minimum_cgpa; minimum_year; maximum_year; all_departments_allowed	Stores academic eligibility rules with range checks.
opportunity_departments	opportunity_id FK; department_id FK	Composite PK lists allowed departments.
opportunity_skills	opportunity_id FK; skill_id FK; requirement_type	Composite PK identifies required or preferred skills.
applications	application_id PK; student_id FK; opportunity_id FK; application_status; applied_at	Unique student and opportunity pair. Tracks saved through final states.
application_status_history	history_id PK; application_id FK; old_status; new_status; changed_by FK; change_note	Records actor time and change. Direct UPDATE and DELETE are rejected by new triggers.

Application state model

The schema defines saved, preparing, applied, shortlisted, selected, rejected and withdrawn states. A future backend must define allowed transitions, check current ownership and eligibility, lock competing updates, write history and notifications together, and reject duplicate or closed deadline submissions. Enum values alone do not implement these business rules.

Documents operations and security tables

Table	Keys and selected fields	Purpose
document_types	document_type_id PK; document_name; description	Reusable document definitions such as resume or NOC.
opportunity_documents	opportunity_id FK; document_type_id FK; is_mandatory	Composite PK identifies requested document types.
application_documents	application_id FK; document_type_id FK; document_status; file_name; notes	Composite PK holds document preparation state. A filename column does not provide an upload pipeline.
preparation_tasks	task_id PK; application_id FK; task_title; due_date; task_status; completed_at	Task titles are unique within an application.
notifications	notification_id PK; user_id FK; notification_type; title; message; related opportunity and application	Stores deadline application task or system messages.
auth_sessions	session_hash PK; user_id FK; created_at; expires_at; revoked_at	Adds revocable sessions. Only a token digest is stored server side.
security_rate_limits	bucket_key PK; window_start; hits	Adds shared attempt counters. Bucket keys are HMAC digests rather than raw identities.

Security migration

The additive security migration creates the final two tables and two rejecting history triggers. It does not drop or reset existing tables. The migration script separates privileged setup credentials from Flask runtime credentials and provisions a dedicated local account with only the table operations needed by current routes.

Data retention boundary

Direct history edits are blocked, and the runtime account has no permission to change that table. Existing foreign key cascades remain. A privileged parent deletion can bypass child history triggers, so these controls do not provide retention against a fully privileged database administrator. No public delete route or runtime delete privilege is supplied. A future deletion feature must address this explicitly.

Implemented workflows and validation

Registration and login

Registration validates name length from 2 to 100 characters, UID format, matching CU email, a 10 to 128 character password and confirmation. A transaction inserts only a student account with a Werkzeug password hash. Duplicate email or UID feedback is safe, and database uniqueness resolves concurrent conflicts.

Login accepts UID or university email and retains compatibility with the old email form field. Email lookup is case insensitive. UID lookup joins the student profile, with a CU email fallback before profile completion. Unknown accounts and wrong or inactive credentials receive the same generic login error. Password verification uses check_password_hash, including a dummy hash for eligible unknown account lookup paths.

Academic profile setup

The route requires a currently active student. It associates the new profile with the authenticated user, not a submitted owner ID. The department must exist, academic year must be 1 to 4, optional CGPA must be finite and between 0 and 10, and optional graduation year must be current year through ten years ahead. UID must match the account email. Existing completed profiles retain their setup completed redirect.

Session lifecycle

After successful login, a random token is generated and its SHA 256 digest is inserted with an eight hour absolute expiry. Previous cookie state is cleared and a new CSRF value is issued. Protected requests look up the digest and current account. Logout is a CSRF protected POST that revokes the record and clears the cookie. Replaying a copied cookie after logout is rejected.

Path	Methods	Access
/	GET	Public
/register	GET and POST	Public registration
/login	GET and POST	Public authentication
/profile/setup	GET and POST	Active student
/logout	POST	CSRF protected session action
/database-test	GET	Active administrator

Security design and audit improvements

The defensive review found two High, five Medium and two Low findings in the original application. Debug mode and global MySQL root usage were the High findings. The fixes were applied without resetting account passwords, rewriting completed profiles or deleting existing records.

Control	Implemented protection
Debugger and errors	Waitress and explicit debugger or reloader disablement. Safe HTTP database and unexpected errors; logs omit exception details and request secrets.
Database least privilege	Dedicated non root account with narrow table specific rights. Migration credentials are separate from runtime configuration.
CSRF and authorization	Tokens on all POST forms; unsafe method checks. Current account and role from MySQL protect profile and diagnostic access.
Password and session controls	Werkzeug hashes; no public admin registration; revocable token digests and absolute expiry; cookie flags appropriate to environment.
Rate limiting	Login 30 attempts per minute per direct IP and 8 per account. Registration 20 per hour per direct IP. Shared atomic MySQL counters.
Browser and production policy	CSP no sniff frame denial referrer and permissions policies and no store. Production requires trusted hosts HTTPS non root credentials and Secure cookies.

Important security limits

The review found no SQL injection in inspected queries and no unsafe template or JavaScript rendering sink. This is evidence about the reviewed code, not a guarantee that the system has no vulnerabilities. File uploads and most opportunity workflows do not exist here and cannot be described as secured features.

Private environment files have restricted Windows access. However, the local migration credential file remains readable by the same operating system user running Flask. Production must separate that secret under a different operations identity or secret store. Local ACL changes also do not remove prior OneDrive copies. CU identity validation confirms format, not email ownership.

Testing and verification results

The final recorded run on 3 November 2026 passed all 16 automated test methods with MySQL tests enabled and no skips. The suite uses generated test identities against the reviewed local schema. Only those fixture accounts are deactivated afterward; records are retained for auditability. The counts below describe verified tests, not unimplemented product features.

Test group	Coverage	Result
Identity and authentication	Matching or invalid CU identity password hashes duplicates UID or email login wrong password and inactive accounts	Pass
Authorization and ownership	Student and admin separation current role recheck forged profile owner and protected diagnostic	Pass
Session and CSRF	Token rotation copied cookie rejection after logout malformed or absent CSRF and POST logout	Pass
Validation and concurrency	Invalid numbers NaN Infinity foreign IDs and duplicate account or profile races	Pass
Rendering and errors	Escaped script like text safe flash rendering no unsafe sinks and sanitized 500 or 503 errors	Pass
Configuration and limits	Production guards trusted hosts cookie flags headers request size and shared rate counters	Pass
Database controls	Non root minimal grants transaction isolation and history trigger presence	Pass

Live and dependency verification

Eight live HTTP checks passed on localhost port 5000: homepage with headers, login page, registration page, logged out profile protection, CSRF rejection, UID login with academic setup access, student denial from diagnostics, and POST logout followed by protected route rejection.

PyPA pip audit reported zero known advisories and zero skipped installed packages. pip check reported no broken requirements. These results are time specific and do not cover unknown vulnerabilities or certify the operating system, MySQL deployment or external infrastructure. Production TLS and mobile visual review were not performed during the security pass.

Deployment limitations and future work

Local operation

The selected application runs from Documents\CampusOpportunityPlanner. Install the pinned requirements in its virtual environment, retain private runtime configuration and launch start.ps1. The preferred server command is shown below. A separate privileged operation applies database/migrations/001_security.sql and provisions runtime permissions; Flask must not use root for routine requests.

.\.venv\Scripts\waitress-serve.exe --listen=127.0.0.1:5000 app:app

Current limitations

Opportunity discovery, live eligibility and readiness calculations, applications, document uploads, preparation planning, notifications and admin management screens remain unimplemented. The landing page readiness percentage is a static illustration. Academic values are self reported, email ownership is not verified, and profile setup is not an editing interface. The system has no password recovery or MFA workflow.

Future development

The next phase should implement authenticated opportunity discovery and an explainable academic eligibility service. A later phase should add application transitions, history and notifications in one transaction, with deadline checks and per resource ownership controls. Document readiness, safe authenticated uploads, preparation tasks and administrative analytics can follow. Each new route needs validation, CSRF, role and ownership tests; schema availability is not sufficient.

A proposed transparent readiness formula could allocate 40 percent to academic eligibility, 30 percent to required skills, 20 percent to required documents and 10 percent to linked task completion. This is a future design option, not a current calculation or a trained machine learning model. Production work should also add verified identity integration, reviewed retention, backups, monitoring and a tested TLS proxy boundary.

Conclusion

The project now has a validated and security hardened account and academic profile foundation with a normalized relational model for expansion. Passing tests demonstrate the implemented controls and workflows. Completing the opportunity and application modules remains necessary before the application can function as a full campus tracker.

References

Project sources: app.py, database.py, security.py, database/schema.sql, database/migrations/001_security.sql, tests/test_security.py and the recorded audit outputs. Technical guidance: flask.palletsprojects.com/en/stable/web-security/; dev.mysql.com/doc/refman/8.0/en/; werkzeug.palletsprojects.com; github.com/pypa/pip-audit. Implementation and verification evidence is recorded in SECURITY_AUDIT.md and SECURITY_TEST_CHECKLIST.md.